Bayview Application

Deployment & Configuration Documentation

Document Version:	**1.0**
Date:	February 25, 2026
Application:	Bayview STS Cabin Kiosk System
Environment:	Pre-Production & Production

Document History

This document and its contents have been prepared and are intended solely as information for Terminals, Global IT, SD&I and use in relation to Terminal Infrastructure Platform Refresh Project – TC1

Version No	Revision Comments	Author	Approved by	Date
1.0		BIH001		February 25, 2026

Table of Contents

- 1. Overview
- 2. System Architecture
- 2.1 Database Layer (XPS_Event.py)
- 2.2 API Layer (n4api.py)
- 2.3 Application Layer (bayview.py)
- 2.4 Main Application (main.py)
- 3. Deployment Steps
- 3.1 Pre-requisites
- 3.2 Deployment Process
- 4. Configuration
- 4.1 Environment Variables
- 4.2 Database Configuration
- 4.3 Network Configuration
- 5. File Structure
- 6. API Endpoints
- 7. Database Schema
- 8. Troubleshooting

1. Overview

The Bayview application is a real-time ship-to-shore (STS) crane monitoring system designed for terminal operations. It provides crane operators with live visibility into container work instructions, vessel bay plans, and move sequences through cabin kiosks deployed at each quay crane (QC).

Key Features

- Real-time work instruction display for QC operators
- Live bay plan visualization with container positions
- Integration with N4 terminal operating system
- Support for 10 quay cranes (QCZ1-QCZ10)
- Automatic IP-based crane identification
- SQLite-based local caching for performance
- Server-sent events (SSE) for live updates

Technology Stack

Component	Technology
Backend Framework	Starlette (Python)
Database	Microsoft SQL Server (N4), SQLite (Local Cache)
API Protocol	SOAP, REST, Server-Sent Events
Frontend	HTML5, JavaScript
Deployment	Python Virtual Environment

2. System Architecture

The Bayview system follows a multi-layered architecture with clear separation of concerns:

2.1 Database Layer (XPS_Event.py)

The database layer manages all data persistence and retrieval operations through two main classes:

WiDatabase Class

- SQLite-based local storage for work instructions
- Separate table per QC crane (QCZ1-QCZ10)
- Async operations for non-blocking I/O
- Automatic table creation on initialization
- UPSERT operations for real-time updates
- Move state tracking (current_move field)

Key Methods:

Method	Purpose
create_table()	Creates QC-specific table if not exists
insert_or_update()	Upserts work instructions with move tracking
read_data()	Retrieves active work instructions for display
update_current_move()	Marks container as currently being moved
delete_comfirmed()	Removes completed moves

XPSEventDatabase Class

Manages connections to the N4 SQL Server database and orchestrates data synchronization.

- ODBC connection to N4 SQL Server
- Continuous polling loop for real-time updates
- Complex SQL queries joining multiple N4 tables
- Vessel point-of-work (POW) monitoring
- Move event tracking and state management
- Background thread for async database operations

Note: The readWi() method runs in a continuous loop, polling N4 every 5 seconds to fetch updated work instructions for all active QC cranes.

2.2 API Layer (n4api.py)

Handles SOAP communication with the N4 Apex web service to retrieve vessel bay models.

Key Functions

get_ship_bin_model(): Sends SOAP request to retrieve vessel class configuration

process_xml_data(): Parses XML response and converts to JSON format

request_bin_model(): Orchestrates the complete bin model retrieval process

SOAP Configuration

Endpoint	**http://10.212.12.146:9081/apex/services/argobasicservice**
Method	basicInvoke
Scope	APMT/MAPTM/PTM/PTM
Authentication	Basic Auth (Base64 encoded)
Timeout	30 seconds

Extracted Data Structures

- Ship Models: Vessel geometry and reference points
- Bin Name Tables: Bay/row/tier naming conventions
- Ship Blocks: Structural bay groupings
- Cell Sections: Row/column definitions
- Cell Stacks: Individual container slot positions with coordinates

2.3 Application Layer (bayview.py)

The BayView class implements the web application logic and HTTP endpoint handlers.

Core Functionality

QC IP Mapping: The system automatically identifies which crane is accessing the application based on the client IP address. IP ranges 10.212.141-150.12 map to QCZ1-QCZ10.

Route Handlers

Route	Method	Purpose
/OT/STS/bayView	GET	Main UI page for crane operators
/OT/STS/bayView/event/{qcId}	GET	SSE stream for work instructions
/OT/STS/ship-bin-model	POST	Retrieve vessel bay configuration
/OT/STS/bayView/clock	GET	SSE stream for synchronized clock
/OT/STS/bay-plan/{vessel}	GET	Vessel stowage plan XML

Server-Sent Events (SSE): The application uses SSE for real-time push updates to client browsers, eliminating the need for polling. The xps_event_stream() method continuously sends updated work instructions every 2 seconds.

2.4 Main Application (main.py)

The main.py file bootstraps the Starlette application and configures middleware.

- Starlette app initialization with debug mode
- CORS middleware configuration for cross-origin requests
- Route registration for all Bayview endpoints
- Template engine setup for HTML rendering
- Logger initialization for application monitoring

3. Deployment Steps

3.1 Pre-requisites

Item	Requirement	Notes
Operating System	Ubuntu 24.04 LTS	Server: SCRBIMMCMATNG02
Python Version	Python 3.8+	With venv support
Database Access	SQL Server 2016+	ODBC Driver 17
Network Access	OT zone network	Access to N4 database
User Permissions	root or sudo	For system packages
Disk Space	Minimum 5GB	For application and logs

Required System Packages

```
# Update package lists
sudo apt update

# Install ODBC driver for SQL Server
sudo apt install -y unixodbc unixodbc-dev

# Install SQL Server ODBC Driver 17
curl https://packages.microsoft.com/keys/microsoft.asc | sudo apt-key add -
curl https://packages.microsoft.com/config/ubuntu/24.04/prod.list | \
    sudo tee /etc/apt/sources.list.d/mssql-release.list
sudo apt update
sudo ACCEPT_EULA=Y apt install -y msodbcsql17

# Verify installation
odbcinst -j
```

3.2 Deployment Process

Step 1: Clone Repository

Clone the application repository to the deployment server.

```
cd /home/Projects/OT
git clone https://github.com/brahim-iharraren/BayView.git.
```

Step 2: Create Virtual Environment

Set up Python virtual environment for dependency isolation.

```
python3 -m venv venv
source venv/bin/activate
```

Step 3: Install Dependencies

Install required Python packages from requirements.txt.

```
pip install --upgrade pip
pip install -r requirements.txt
```

Step 4: Configure Environment

Set up .env file with database credentials and configuration.

```
cat > .env << EOF
N4_DB_HOST=10.212.12.146
N4_DB_PORT=1433
N4_DB_NAME=sparcsn4
N4_DB_USER=your_username
N4_DB_PASS=your_password
EOF

chmod 600 .env
```

Step 5: Initialize Database

Create SQLite database and verify N4 connectivity.

Step 6: Test Application

Run application in test mode to verify all components.

Step 7: Configure Service

Set up systemd service for automatic startup.

Step 8: Start Service

Enable and start the Bayview service.

4. Configuration

4.1 Environment Variables

The application uses environment variables for configuration management. All sensitive credentials should be stored in the .env file, which is loaded using python-dotenv.

Variable	Purpose	Example
N4_DB_HOST	N4 database server IP	10.212.12.146
N4_DB_PORT	Database port number	1433
N4_DB_NAME	Database name	sparcsn4
N4_DB_USER	Database username	bih00133
N4_DB_PASS	Database password	[REDACTED]

4.2 Database Configuration

The system connects to two databases:

Primary Database (N4 SQL Server): Source of truth for all terminal operations data

- Host: 10.212.12.146:1433
- Database: sparcsn4
- Driver: ODBC Driver 17 for SQL Server
- Connection pooling: Enabled
- Tables accessed: inv_wi, xps_che, xps_ecevent, inv_move_event, xps_pointofwork

Local Cache (SQLite): High-performance local storage for active work instructions

- Location: ./wi.db (local filesystem)
- Tables: QCZ1 through QCZ10 (one per crane)
- Primary key: uyv_gkey (unique visit gkey)
- Indexes: Automatically created on primary keys
- Backup: Not required (regenerated from N4)

4.3 Network Configuration

The application requires specific network access for proper operation:

Component	Network	Protocol	Port
Crane Kiosks	10.212.141-150.12	HTTP	80/443
N4 Database	10.212.12.68	TCP/SQL	1433
N4 SOAP API	10.212.12.146	HTTP/SOAP	9081
Application Server	10.212.12.100	HTTP/SSE	80/443

Required Firewall Rules

- Allow incoming HTTP (8000) from 10.212.141-150.12 (crane kiosks)
- Allow outgoing TCP (1433) to 10.212.12.68 (N4 database)
- Allow outgoing HTTP (9081) to 10.212.12.146 (N4 SOAP API)
- Allow SSH (22) from management network for administration

5. File Structure

The application follows a modular structure with clear separation of concerns:

```
/home/Projects/OT/
├── main.py                          # Application entry point
├── bayview_utils/                   # Core application modules
│   ├── __init__.py
│   ├── bayview.py                   # Web application logic
│   ├── XPS_Event.py                 # Database layer
│   ├── n4api.py                     # N4 SOAP API client
│   └── ship_bin_model/              # Cached vessel configurations
│   |     └── [vessel_ref].json
│   └── database/                    # Cached data
│         └── wi.db
├── templates/                       # HTML templates
│   ├── bayView.html                 # Main crane operator UI
│   └── bayplan.html                 # Bay plan viewer
├── static/                          # Static assets (CSS, JS, images)
├── venv/                            # Python virtual environment
├── wi.db                            # SQLite local cache
├── .env                             # Environment configuration
├── requirements.txt                 # Python dependencies
├── myLogger.py                      # Logging configuration
└── logs/                            # Application logs
    └── bayview_[date].log
```

Key Files Description

File	Purpose
main.py	Application bootstrap, route registration, CORS config
bayview.py	HTTP handlers, SSE streams, IP-to-QC mapping
XPS_Event.py	Database models, N4 queries, data synchronization
n4api.py	SOAP client for vessel bay model retrieval
myLogger.py	Centralized logging configuration
wi.db	SQLite cache of active work instructions
.env	Sensitive configuration (DB credentials)

6. API Endpoints

The Bayview application exposes several HTTP endpoints for client access:

GET /OT/STS/bayView

Purpose: Serves the main crane operator interface

Authentication: IP-based (crane kiosk networks only)

Response: HTML page with embedded QC number

GET /OT/STS/bayView/event/{qcId}

Purpose: Server-Sent Events stream for real-time work instructions

Update Frequency: Every 2 seconds

Response Type: text/event-stream

POST /OT/STS/ship-bin-model

Purpose: Retrieve vessel bay configuration for display

Content-Type: application/json

GET /OT/STS/bayView/clock

Purpose: Synchronized clock display for all kiosks

Update Frequency: Every 1 second

7. Database Schema

The SQLite local cache maintains one table per QC crane with the following key columns:

QCZ[1-10] Table Schema

Column	Type	Description
timestamp	DATETIME	Record creation/update time
vessel_ref	TEXT	Vessel class reference
pow_name	TEXT	Point of work (QCZ1-QCZ10)
che_name	TEXT	Container handling equipment
unit_id_name	TEXT	Container number
current_move	INTEGER	Move state (0=queued, 1=active, 2=completed)
sequence	TEXT	Work instruction sequence number
pos_slot	TEXT	Bay/row/tier position
move_kind	TEXT	LOAD/DSCH/REHN/SHFT
uyv_gkey	INTEGER	Unique visit key (PRIMARY KEY)
wi_gkey	TEXT	Work instruction key
itv_gkey	TEXT	Equipment visit key
work_queue_gkey	TEXT	Work queue reference

Move State Values

Value	State	Description
0	Queued	Work instruction in queue, not yet started
1	Active	Container currently being moved
2	Completed	Move finished, will be deleted on next cycle

N4 SQL Server Tables

The application queries the following N4 tables to retrieve work instructions:

inv_wi: Work instructions (container moves to be executed)

xps_che: Container handling equipment status and position

xps_ecevent: Equipment controller events

inv_move_event: Historical move events for tracking

xps_pointofwork: Quay crane point-of-work assignments

inv_unit: Container unit information

inv_unit_fcy_visit: Container facility visit details

vsl_vessel_classes: Vessel class definitions

8. Troubleshooting

Common Issues and Solutions

Issue: No work instructions displayed

Solutions:

1. Verify QC crane is assigned to a vessel in N4
2. Check that pow.qc_vessel_reference is not empty
3. Confirm work queue exists in inv_wi table
4. Review application logs for database errors

Issue: Database connection failure

Solutions:

5. Verify .env file contains correct credentials
6. Test connectivity: telnet 10.212.12.68 1433
7. Check ODBC driver installation: odbcinst -j
8. Confirm user has SELECT permissions on sparcsn4 tables

Issue: SSE connection drops frequently

Solutions:

9. Check network stability between kiosk and server
10. Verify firewall not blocking long-lived connections
11. Review nginx/proxy timeout settings
12. Check application logs for exception traces

Issue: Wrong QC number displayed

Solutions:

13. Verify kiosk IP address in 10.212.141-150.12 range
14. Check X-Real-IP header is set correctly by proxy
15. Review qc_map dictionary in bayview.py
16. Force QC number using URL: /OT/STS/bayView/event/5

Issue: Bay model not loading

Solutions:

17. Verify N4 SOAP API accessible: curl http://10.212.12.146:9081
18. Check vessel_ref is valid in N4 vsl_vessel_classes
19. Review SOAP request/response in logs
20. Confirm ship_bin_model directory is writable

Diagnostic Commands

Use these commands to diagnose application issues:

```
# Check application status
systemctl status bayview

# View recent logs
tail -f /home/Projects/OT/logs/bayview_$(date +%Y%m%d).log

# Test database connectivity
python3 << EOF
import pyodbc
conn_str = "DRIVER={ODBC Driver 17 for SQL
Server};SERVER=10.212.12.68,1433;DATABASE=sparcsn4;UID=user;PWD=pass"
conn = pyodbc.connect(conn_str)
cursor = conn.cursor()
cursor.execute("SELECT COUNT(*) FROM inv_wi")
print(f"Row count: {cursor.fetchone()[0]}")
EOF

# Check SQLite database
sqlite3 wi.db "SELECT name FROM sqlite_master WHERE type='table';"
sqlite3 wi.db "SELECT COUNT(*) FROM QCZ1;"

# Monitor SSE stream
curl -N http://localhost:8000/OT/STS/bayView/event/1
```

Support Contacts

For additional support, contact the following teams:

Application Team: BIH IT Team

Database Team: sparcsN4 Database Administrators

Network Team: Terminal Network Operations

QC Operations: Equipment Maintenance Supervisor